

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ: ИССЛЕДОВАНИЕ РАБОТЫ СЕГМЕНТА СЕТИ CSMA (ВАРИАНТ 4)

Найденова Дария
Сагдиев Айрат

321 группа

ЦЕЛЬ РАБОТЫ

Изучить работу сегмента сети CSMA при различных значениях интенсивности генерации пакетов α и различном числе станций N , исследовать поведение очередей адаптеров, занятость канала и потери пакетов в условиях отсутствия коллизий.

ПОСТАНОВКА ЗАДАЧИ

- Для модели сети CSMA со следующими параметрами:
- Скорость канала: $B=100$ Мбит/с
- Длина канала: $L=30$ м
- Длина пакета: $l=1500$ байт
- Экспоненциальное распределение интервалов между пакетами со средним $1/\alpha$
- Время моделирования: 10 секунд (для первой части) и до 60 секунд (для второй части)
- Требуется:
 1. Найти максимальное количество станций N , при котором занятость канала не превышает 50% для значений $\alpha=10, 50, 100$.
 2. Найти максимальное N , при котором очередь адаптера остаётся стабильной (не растёт бесконечно).

ОПИСАНИЕ МОДЕЛИ

- Топология: N хостов подключены к одному общему каналу CSMA.
- Канал: CSMA без коллизий, скорость 100 Мбит/с
- Трафик: Каждый хост генерирует пакеты с экспоненциальным интервалом "Exp" (α).
- Буфер адаптера: В первой части ограничен 20 пакетами, во второй — неограничен.
- Механизм передачи: Хост проверяет занятость канала перед отправкой. Если канал свободен — пакет отправляется, иначе — ожидание.

РЕАЛИЗАЦИЯ МОДЕЛИ

1. РЕАЛИЗАЦИЯ ГЕНЕРАЦИИ ПАКЕТОВ

Необходимо создать механизм поступления трафика в систему, имитирующий реальную работу сетевых приложений.

- **Экспоненциальное распределение интервалов** — каждый хост генерирует пакеты с промежутками, подчиняющимися экспоненциальному закону со средним значением $1/\alpha$. Это соответствует пуассоновскому потоку — классической модели сетевого трафика.
- **Уникальная идентификация** — каждый пакет получает уникальный числовой идентификатор (`global_pkt_id`), что позволяет отслеживать его жизненный цикл от создания до завершения передачи.
- **Планирование событий прибытия** — используется подход дискретно-событийного моделирования: при создании пакета сразу планируется следующее событие прибытия, образуя цепочку:
 - Прибытие пакета → обработка → планирование следующего прибытия
- **Интеграция с CSMA-логикой** — генерация не является независимым процессом; при создании пакета сразу проверяются условия CSMA для возможной немедленной передачи.

2. РЕАЛИЗАЦИЯ БУФЕРА АДАПТЕРА (ОЧЕРЕДИ ПАКЕТОВ)

Необходимо создать механизм временного хранения пакетов, ожидающих доступа к разделяемому каналу.

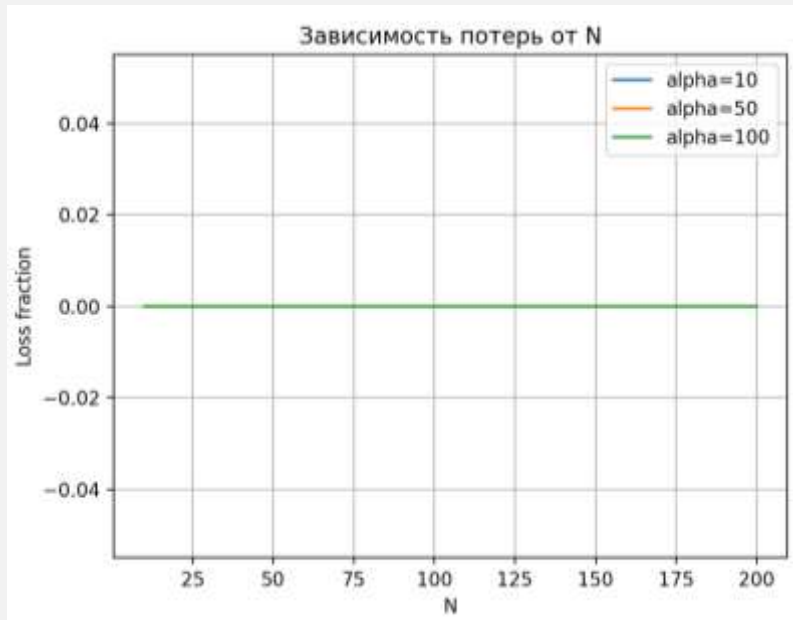
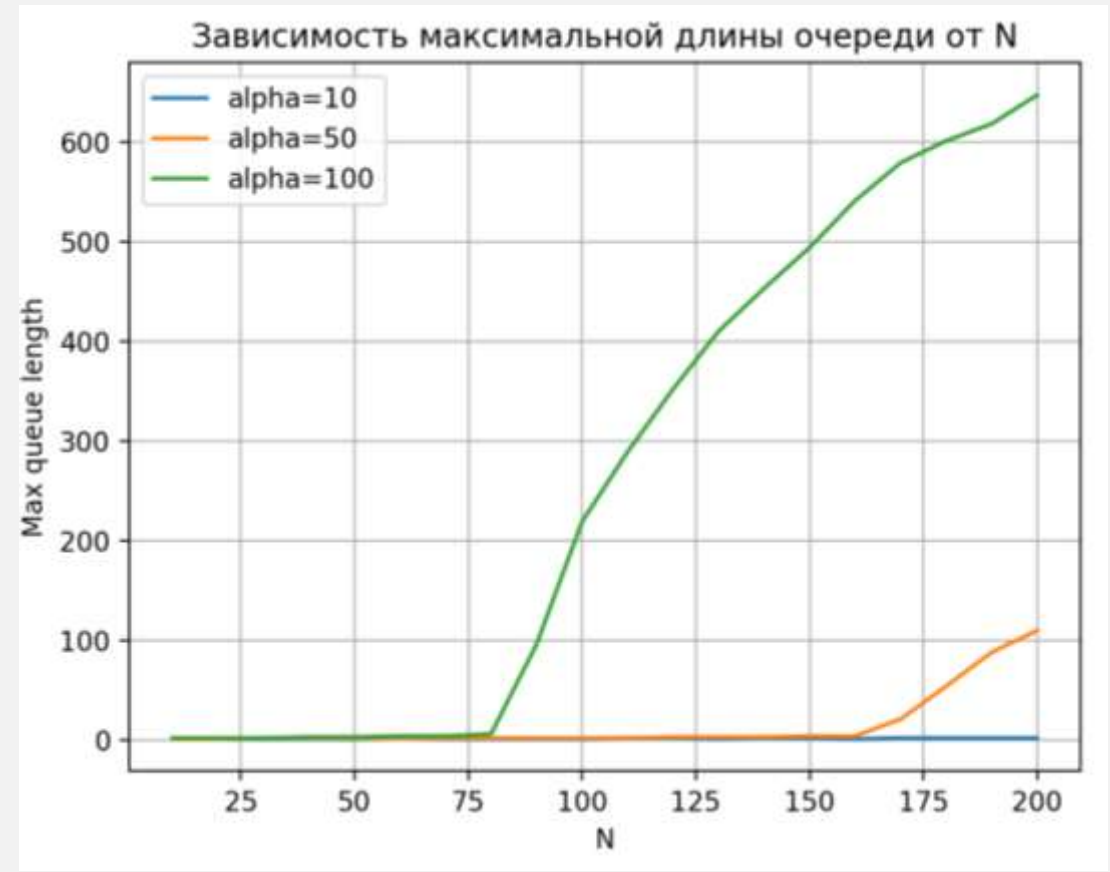
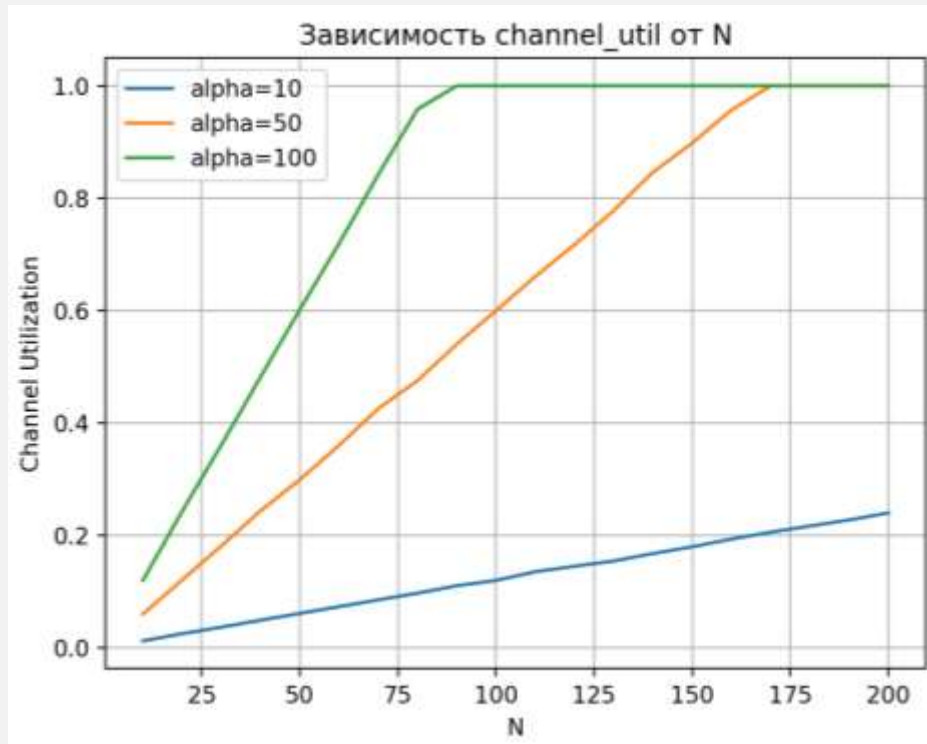
- **Распределённые очереди** — у каждого хоста своя отдельная очередь (`deque<Packet>`), что соответствует архитектуре реальных сетевых адаптеров, где каждый интерфейс имеет собственный буфер.
- **Статистика очереди** — отслеживаются ключевые метрики:
 - **Максимальная длина** — пиковая нагрузка на буфер
 - **Средняя длина** (time-weighted) — интеграл длины очереди по времени, делённый на время симуляции
 - **Площадь под кривой** — `queue_time_area` для точного расчёта средних
- **Механизм потерь** — при включённом ограничении и переполнении очереди пакет отбрасывается с увеличением счётчика `total_dropped`.

3. МОДЕЛИРОВАНИЕ ПРОЦЕССА ПЕРЕДАЧИ (CSMA ЛОГИКА)

Необходимо: Реализовать протокол доступа к разделяемой среде передачи без коллизий.

Что было реализовано:

- **Двухфазный цикл работы:**
 - Проверка доступности канала при прибытии пакета
 - Передача данных в течение фиксированного времени
 - Освобождение канала и выбор следующего передатчика
- **Алгоритм CSMA**
- **Механизм выбора следующего передатчика** — после завершения передачи реализован алгоритм выбора пакета с самым ранним временем прибытия (FIFO по времени создания).
- **Учёт занятости канала** — точное измерение времени, когда канал используется для передачи:
 - Фиксация времени начала передачи (`channel_busy_start`)
 - Накопление суммарного времени занятости (`cumulative_channel_busy`)
 - Вычисление утилизации: $\text{channel_util} = \text{cumulative_channel_busy} / \text{simTime}$

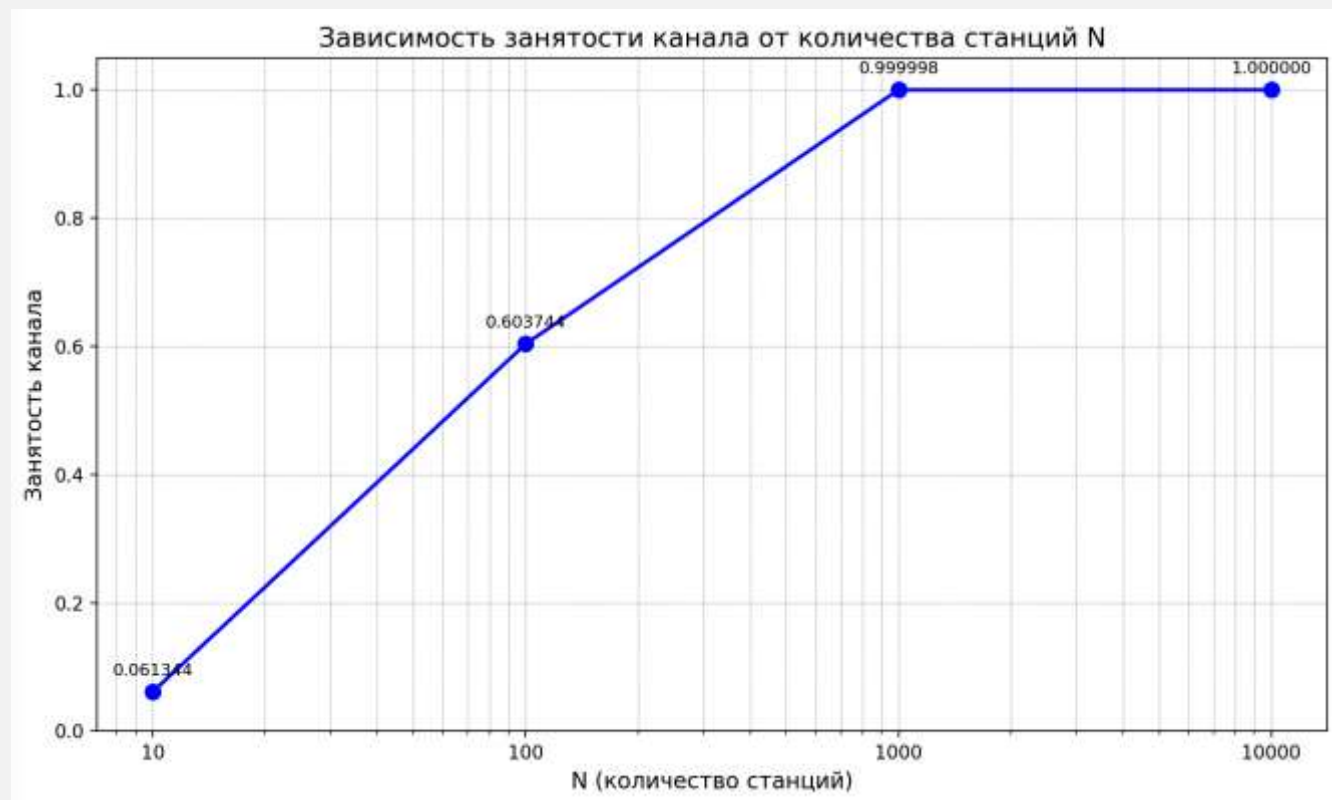


ЗАМЕР ХАРАКТЕРИСТИК

Характеристики, которые будет вычислять модель:

- Средняя доля потерянных пакетов
- Среднее число попыток повторной передачи пакета
- Среднее и максимальное число пакетов в буфере адаптера
- Средняя занятость канала

Будем проводить замеры при различных N со такими параметрами: $\alpha = 50$, $B=100$ Мбит/с, $L=30$ м, $l=1500$ байт.



N (станций)	Хар1: Доля потерь	Хар2: Средние попытки	Хар3: Макс. очередь	Хар4: Занятость канала
10	0.000	1.000	2	0.061344
100	0.000	1.000	2	0.603744
1000	0.000	1.000	497	0.999998
10000	0.000	1.000	591	1.000000

НАХОЖДЕНИЕ МАКСИМАЛЬНОГО N ПРИ $\text{ХАР}_4 \leq 0.5$

- Программа `find_max_n.py` — это **вспомогательный скрипт для автоматизации** поиска максимального количества станций N, при котором занятость канала (channel utilization) не превышает заданный порог (по умолчанию 50%).
- Формирует командную строку с параметрами
- Выполняет симуляцию через `subprocess`
- Извлекает значение **занятости канала** из вывода с помощью регулярного выражения
- Использует **алгоритм бинарного поиска** для эффективного нахождения границы
- Проверяет значения N от 1 до 500 (по умолчанию)
- Сохраняет последнее N, при котором выполняется условие: `channel utilization < 0.5`

НАХОЖДЕНИЕ МАКСИМАЛЬНОГО N ПРИ $\text{ХАР}_4 \leq 0.5$

Поиск максимального N при channel_util <= 0.5 для alpha=10.0

N=250, util=0.2989
N=375, util=0.4510
N=438, util=0.5284
N=406, util=0.4903
N=422, util=0.5067
N=414, util=0.4978
N=418, util=0.5047
N=416, util=0.4994
N=417, util=0.4998

=====
Максимальное N = 417
channel util = 0.49985

Поиск максимального N при channel_util <= 0.5 для alpha=100.0

N=250, util=1.0000
N=125, util=1.0000
N=62, util=0.7435
N=31, util=0.3752
N=46, util=0.5497
N=38, util=0.4549
N=42, util=0.5037
N=40, util=0.4809
N=41, util=0.4913

=====
Максимальное N = 41
channel_util = 0.49134

- Для $\alpha = 10$: N = 417
- Для $\alpha = 100$: N = 41

В обоих случаях средняя доля потерянных пакетов равна нулю.

НАХОЖДЕНИЕ МАКСИМАЛЬНОГО N ПРИ $\text{ХАР}_4 \leq 0.5$

```
Поиск максимального N при channel_util <= 0.5 для alpha=50.0
N=250, util=1.0000
N=125, util=0.7497
N=62, util=0.3710
N=93, util=0.5584
N=77, util=0.4624
N=85, util=0.5127
N=81, util=0.4888
N=83, util=0.5019
N=82, util=0.4918
=====
Максимальное N = 82
channel_util = 0.49183
```

- Для $\alpha = 50$: Максимум 83 хоста могут работать одновременно
 - При этом канал будет примерно 50%
 - При 84 хостах канал перегружается (занятость > 50%)
- Следовательно, $N = 83$, при этом Хар_1 (ср. доля потерянных пакетов) = 0

НАХОЖДЕНИЕ МАКСИМАЛЬНОГО N С ОГРАНИЧЕННОЙ ОЧЕРЕДЬЮ

- Написанная программа `find_stable_queue.py` находит максимальное количество хостов (N), при котором очередь стабильна (не растёт неограниченно), используя бинарный поиск.

НАХОЖДЕНИЕ МАКСИМАЛЬНОГО N С ОГРАНИЧЕННОЙ ОЧЕРЕДЬЮ

Поиск максимального стабильного N для $\alpha=10.0$

Проверка N=250...

N=250: $\max Q(20)=2$, $\max Q(40)=2$

Проверка N=375...

N=375: $\max Q(20)=2$, $\max Q(40)=2$

Проверка N=438...

N=438: $\max Q(20)=2$, $\max Q(40)=2$

Проверка N=469...

N=469: $\max Q(20)=2$, $\max Q(40)=2$

Проверка N=485...

N=485: $\max Q(20)=2$, $\max Q(40)=2$

Проверка N=493...

N=493: $\max Q(20)=2$, $\max Q(40)=3$

Проверка N=497...

N=497: $\max Q(20)=3$, $\max Q(40)=2$

Проверка N=499...

N=499: $\max Q(20)=2$, $\max Q(40)=3$

Проверка N=500...

N=500: $\max Q(20)=2$, $\max Q(40)=2$

=====

Максимальное стабильное N = 500

Для $\alpha =$
10: N = 500

Для $\alpha =$
100: N = 83

Поиск максимального стабильного N для $\alpha=100.0$

Проверка N=250...

N=250: $\max Q(20)=1464$, $\max Q(40)=2800$

Проверка N=125...

N=125: $\max Q(20)=739$, $\max Q(40)=1421$

Проверка N=62...

N=62: $\max Q(20)=3$, $\max Q(40)=4$

Проверка N=93...

N=93: $\max Q(20)=248$, $\max Q(40)=462$

Проверка N=77...

N=77: $\max Q(20)=5$, $\max Q(40)=5$

Проверка N=85...

N=85: $\max Q(20)=68$, $\max Q(40)=109$

Проверка N=81...

N=81: $\max Q(20)=8$, $\max Q(40)=8$

Проверка N=83...

N=83: $\max Q(20)=18$, $\max Q(40)=14$

Проверка N=84...

N=84: $\max Q(20)=24$, $\max Q(40)=41$

=====

Максимальное стабильное N = 83

НАХОЖДЕНИЕ МАКСИМАЛЬНОГО N С ОГРАНИЧЕННОЙ ОЧЕРЕДЬЮ

Поиск максимального стабильного N для $\alpha=50.0$

Проверка N=250...

N=250: $\max Q(20)=393$, $\max Q(40)=730$

Проверка N=125...

N=125: $\max Q(20)=3$, $\max Q(40)=3$

Проверка N=187...

N=187: $\max Q(20)=132$, $\max Q(40)=271$

Проверка N=156...

N=156: $\max Q(20)=6$, $\max Q(40)=5$

Проверка N=171...

N=171: $\max Q(20)=41$, $\max Q(40)=82$

Проверка N=163...

N=163: $\max Q(20)=6$, $\max Q(40)=6$

Проверка N=167...

N=167: $\max Q(20)=13$, $\max Q(40)=14$

Проверка N=169...

N=169: $\max Q(20)=32$, $\max Q(40)=45$

Проверка N=168...

N=168: $\max Q(20)=18$, $\max Q(40)=34$

=====
Максимальное стабильное N = 167

- Для $\alpha = 50$: максимум 166 хостов можно поддерживать со стабильной очередью
- При 167+ хостах очередь начинает расти неограниченно
- Для данной нагрузки ограничивающим фактором является занятость канала, а не стабильность очереди. Система может поддерживать больше хостов со стабильной очередью, чем при ограничении по занятости канала.

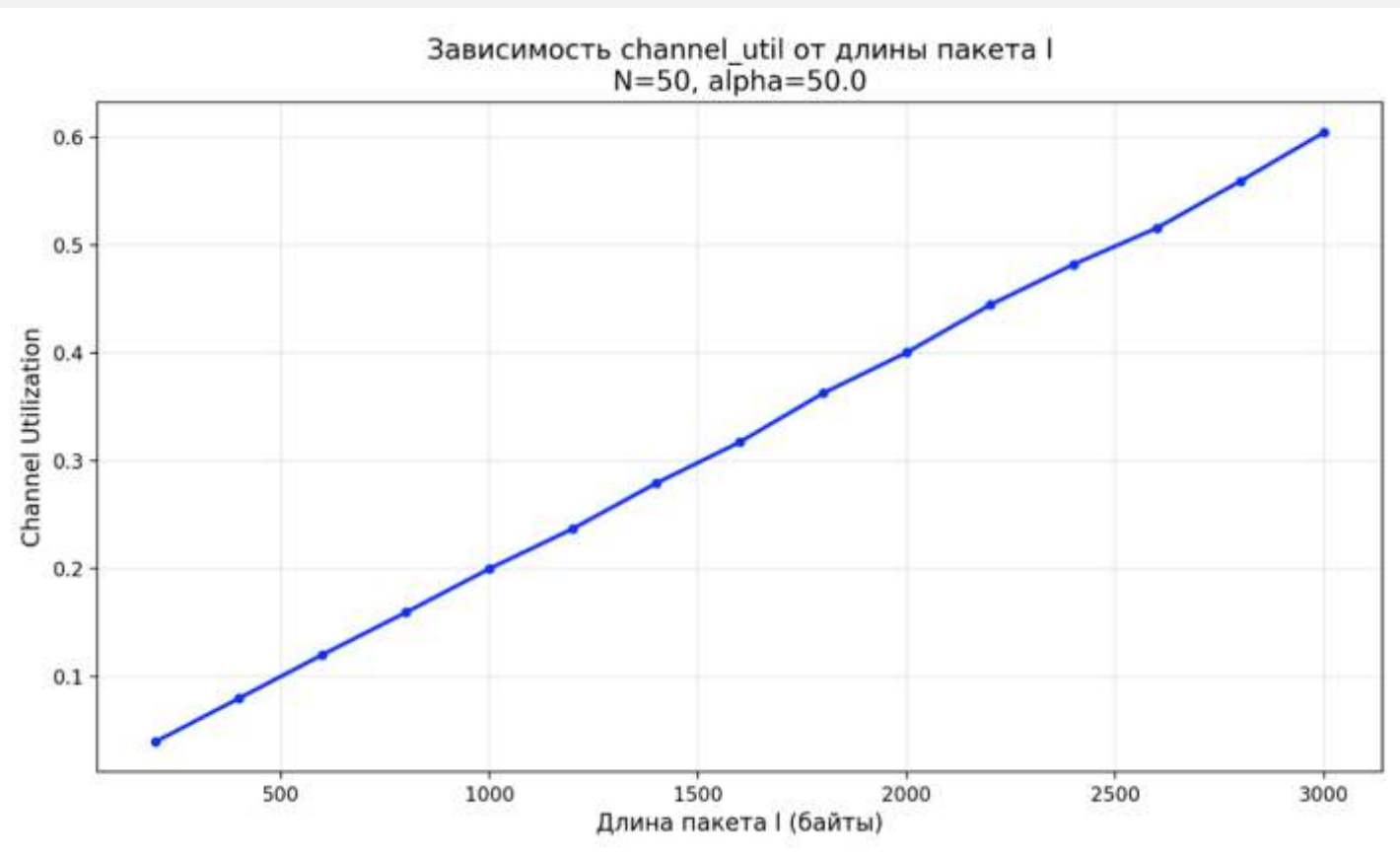
ДОПОЛНИТЕЛЬНОЕ ИССЛЕДОВАНИЕ

Исследование зависимости средней занятости канала (`channel_util`) от длины пакета `l` (в байтах).

- **Основные функции программной реализации:**

1. `run_sim(N, alpha, l_bytes, ...)` — запускает симуляцию (`csma_sim.cpp`) с заданными параметрами и извлекает `channel_util`
2. `analyze_channel_util_vs_l(N, alpha, l_min, l_max, l_step)` — анализирует зависимость для диапазона значений `l`
3. `plot_results(...)` — строит график зависимости

РЕЗУЛЬТАТЫ



РЕЗУЛЬТАТЫ

- Прямая зависимость: `channel_util` линейно растёт с увеличением `l`
- Пропорциональность: примерно пропорциональна длине пакета
- при $l \approx 2500$ байт достигается $\text{channel_util} \approx 50\%$ для данных параметров

РЕЗУЛЬТАТЫ

- **Длина пакета является значимым параметром управления производительностью CSMA-сегмента сети.** Линейная зависимость занятости канала от длины пакета позволяет **точно прогнозировать и регулировать нагрузку** на сеть.
- При проектировании и настройке сетей необходимо **учитывать не только количество станций и интенсивность трафика, но и размер передаваемых пакетов.** Оптимизация этого параметра может существенно улучшить эффективность использования сетевых ресурсов без изменения топологии или аппаратного обеспечения.